

AI based multi eye disease ocular diagnosis

Submitted in partial fulfillment of the requirements of the degree of

Bachelor of Engineering

By

Adnan khan pinjari (221702)

Mir wasi Abbas (221727)

Mohammad Yasir Mohammd Iqbal (221728)

Vanu Faisal Abdul Muttalib (221759)

Supervisor:

Prof . Nasheet Tarik



**Department of Computer Science and Engineering
(Artificial Intelligence & Machine Learning)**

Anjuman I Islam's

M.H. SABOO SIDDIK COLLEGE OF ENGINEERING

UNIVERSITY OF MUMBAI

2025-2026

Anjuman I Islam's
M.H. SABOO SIDDIK COLLEGE OF ENGINEERING

Department of Computer Science and Engineering

Artificial Intelligence & Machine Learning



CERTIFICATE

This is to certify that the Major Project entitled “**AI based multi eye disease ocular diagnosis**” is the bonafide work of) submitted to the University of Mumbai in fulfillment of the requirement for the Major Project-II Semester VIII project work of B.E. in Computer Science & Engineering (Artificial Intelligence and Machine Learning) at M.H. Saboo Siddik College of Engineering, Byculla, Mumbai for the academic year 2025-2026.

Guide

Internal Examiner

External Examiner

Prof. Tarannum Shaikh
Project Coordinator

Dr. Irfan Landge
H.O.D (CSE-AIML)

Dr. Shafi Pathan
Principal

Major Project-II Report Approval

This project report entitled “**Ai based multi eye disease ocular diagnosis**” by **Vanu Faisal Abdul Muttalib, Mohammad Yasir, Adnan khan pinjari and Mir wasi abbas** is approved for the Major Project-II Semester VIII project work of B.E Computer Science and Engineering (Artificial Intelligence & Machine Learning) at M.H. Saboo Siddik College of Engineering, Byculla, Mumbai, in the academic year 2025-2026.

Internal Examiner

External Examiner

Date:

Declaration

We declare that this written submission represents our ideas in our own words and where others' ideas or words have been included, we have adequately cited and referenced the original sources. We also declare that we have adhered to all principles of academic honesty and integrity and have not misrepresented or fabricated or falsified any idea, data, fact, or source in our submission. We understand that any violation of the above will be cause for disciplinary action by the Institute and can also evoke penal action from the sources which have thus not been properly cited or from whom proper permission has not been taken when needed.

Adnan khan pinjari (221702)

Mir wasi Abbas (221727)

Mohammad Yasir Mohammd Iqbal (221728)

Vanu Faisal Abdul Muttalib (221759)

Date:

Abstract

Ocular diseases are among the leading causes of permanent vision impairment and blindness worldwide. Early and accurate diagnosis is critical for effective treatment, yet manual examination of retinal fundus images by ophthalmologists remains time-consuming and prone to subjectivity. This project presents an AI-Based Multi Eye Disease Ocular Diagnosis system that leverages deep learning techniques for automated classification of retinal diseases from fundus images.

The proposed system is built upon the Ocular Disease Intelligent Recognition (ODIR-5K) dataset, which contains labelled fundus photographs from both left and right eyes of patients. Three state-of-the-art Convolutional Neural Network architectures — EfficientNetB0, ResNet50, and MobileNetV2 — are trained, fine-tuned, and evaluated on this dataset. The system classifies retinal images into four major categories: Normal, Myopia, Hypertension, and Diabetic Retinopathy.

Pre-processing techniques including resizing, normalization, and data augmentation were applied to improve model generalization. Transfer learning was employed to leverage pre-trained ImageNet weights, accelerating convergence and improving performance on the relatively small medical dataset. Experimental results demonstrate that EfficientNetB0 achieves the highest test accuracy of 91.4%, outperforming ResNet50 (88.7%) and MobileNetV2 (86.2%), while maintaining a low computational footprint.

The developed system is integrated into a web-based Flask application that allows healthcare professionals to upload fundus images and receive instant diagnostic predictions with confidence scores. This work demonstrates the potential of deep learning in supporting clinical decision-making and facilitating early detection of sight-threatening ocular conditions.

Acknowledgement

I take this opportunity to express my deep sense of gratitude to my project guide **Prof. Nasheet Tarik** for her continuous guidance and encouragement throughout the duration of my project work. It is especially because of her experience and wonderful knowledge; I can fulfill the requirement of completing the project within the stipulated time. I also thank **prof. tarannum** our Project Coordinator, for her extreme support in reviewing the project activities. I would also like to thank **Dr. Irfan Landge**, the Head of Department, Computer Science and Engineering (Artificial Intelligence & Machine Learning) for his encouragement and support. I would also like to thank our Principal **Dr. Shafi Pathan** and the **Management of M.H. Saboo Siddik College of Engineering, Byculla** for providing us all the facilities and the friendly work environment. I acknowledge with thanks the assistance provided by departmental staff, library, and lab attendants for their help.

Adnan khan pinjari (221702)

Mir Wasi Abbas (221727)

Mohammad Yasir Mohammd Iqbal (221728)

Vanu Faisal Abdul Muttalib (221759)

Table of Content

Abstract	i
Table of Content	iii
List of Figures	vi
List of Tables	vii
Abbreviation Notation and Nomenclature	viii

Sr. No	Chapter	Page No.
1.	Introduction and Background 1.1. Introduction 1.2. Background 1.3. Motivation 1.4. Problem Statement 1.4.1. Problem Definition 1.4.2. Proposed Work 1.4.3. Proposed Methodology 1.5. Objective and Scope 1.6. Advantages 1.7. Disadvantages	1
2.	Literature Survey 2.1. Introduction 2.2. Research Papers 2.3. Conclusion	6

Sr. No	Chapter	Page No.
3.	System Analysis 3.1. Introduction 3.2. System Design 3.2.1. Introduction to System Planning 3.2.2. Software Design Approach 3.3. Gantt Chart 3.4. Time Line Chart 3.5. Cost Estimation 3.6. Feasibility 3.7. Conclusion	10
4.	System Design 4.1. Introduction 4.2. Block Diagram 4.3. System Architecture 4.4. Data Flow Diagram 4.5. Table Structure 4.6. State Transition Diagram 4.7. E-R Diagram 4.8. Sequence Diagram 4.9. Class Diagram 4.10. Conclusion	25
5.	Implementation 5.1. Introduction 5.2. Algorithm	34

Sr. No	Chapter	Page No.
6.	Results 6.1. Introduction 6.2. Application Workflow Description 6.3. Conclusion	39
7.	Future Development and Conclusion 7.1. Introduction 7.2. Limitations 7.3. Future Enhancement 8.1. Conclusion	48

References 51

Appendix 52

List of Figures

Sr. No	Fig No.	Fig Description	Page No.
1.	3.3	Project Gantt Chart	18
2.	3.4	Gantt Chart (First 15 Weeks)	19
3	4.2	Block Diagram	25
5.	4.3	System Architecture	26
6.	4.4.1	Data Flow Diagram Level 0	27
7.	4.4.2	Data Flow Diagram Level 1	28
8.	4.5	Table Structure	29
9.	4.6	State Transition	30
10.	4.7	E-R Diagram	31
11.	4.8	Sequence Diagram	32
12.	4.9	Class Diagram	33
13.	5.1	ODIR-5K Dataset Class Distribution	36
14.	5.2	Training Hyperparameters	38
15.	6.1	Model Performance Comparison Table	41
16.	6.2	Per-Class Metrics – EfficientNetB0	42
17.	6.3	Confusion Matrix – EfficientNetB0	43
18.	6.4	Training History Summary	44
19.	6.5	Sample Prediction Results	45

List of Tables

Sr. No	Table No.	Table Description	Page No.
1.	2.2	Literature Survey	7
2.	3.5	Cost Estimation	14

Abbreviation and Nomenclature

AI	Artificial Intelligence
ML	Machine Learning
DL	Deep Learning
CNN	Convolutional Neural Network
ODIR	Ocular Disease Intelligent Recognition
DR	Diabetic Retinopathy
EfficientNetB0	Efficient Neural Architecture Baseline 0
ResNet	Residual Network
MobileNet	Mobile Neural Network
API	Application Programming Interface
GPU	Graphics Processing Unit
CPU	Central Processing Unit
RGB	Red Green Blue
ReLU	Rectified Linear Unit
SGD	Stochastic Gradient Descent
Adam	Adaptive Moment Estimation
GAP	Global Average Pooling
BN	Batch Normalization
DFD	Data Flow Diagram
ER	Entity Relationship
UML	Unified Modeling Language
UI	User Interface
Flask	Python Micro Web Framework
HTML	HyperText Markup Language
CSS	Cascading Style Sheets

Chapter 1: Introduction and Background

Content:

- 1.1. Introduction
- 1.2. Background
- 1.3. Motivation
- 1.4. Problem Statement
 - 1.4.1. Problem Definition
 - 1.4.2. Proposed Work
 - 1.4.3. Proposed Methodology
- 1.5. Objective and Scope
 - 1.5.1. Scope
 - 1.5.2. Objectives
- 1.6. Advantages
- 1.7. Disadvantages

1.1. Introduction

Vision is one of the most vital human senses, and eye diseases represent a significant global health burden. According to the World Health Organization, more than 2.2 billion people suffer from vision impairment worldwide, with a substantial portion caused by preventable or treatable conditions such as diabetic retinopathy, myopia, and hypertension-related retinopathy. Timely diagnosis is the cornerstone of effective treatment and prevention of irreversible blindness.

Traditional ocular diagnosis relies on the expertise of trained ophthalmologists who manually examine retinal fundus images to detect disease signs. This process is slow, expensive, and subject to inter-observer variability. With the increasing prevalence of eye diseases and a global shortage of specialist physicians, there is an urgent need for automated, reliable, and scalable diagnostic tools.

This project presents an Artificial Intelligence-based system for multi-disease ocular diagnosis using deep learning and retinal fundus image analysis. The system employs three powerful Convolutional Neural Network (CNN) architectures — EfficientNetB0, ResNet50, and MobileNetV2 — trained on the publicly available ODIR-5K dataset to classify fundus images into four categories: Normal, Myopia, Hypertension, and Diabetic Retinopathy. The resulting model is deployed as an interactive web application, enabling clinicians and screening programs to obtain rapid diagnostic support.

1.2. Background

The retina, located at the back of the eye, provides a window into the vascular and neural health of the human body. Retinal fundus photography captures high-resolution images of the optic disc, macula, and surrounding blood vessels, making it a preferred non-invasive tool for ocular and systemic disease screening.

Diabetic Retinopathy (DR) affects approximately one-third of diabetic patients and is the leading cause of blindness in the working-age population. It manifests as microaneurysms, haemorrhages, and neovascularization on the retinal surface. Myopia, or short-sightedness, is one of the most prevalent refractive errors globally and can lead to retinal degeneration in severe cases. Hypertensive Retinopathy results from chronic high blood pressure affecting retinal vessels, causing narrowing, nicking, and haemorrhage. Early detection of these conditions through retinal imaging enables preventive intervention.

Deep learning, especially CNNs, has revolutionized medical image analysis. Models trained on large annotated datasets can learn hierarchical feature representations directly from pixel data, achieving performance comparable to or exceeding human specialists. Transfer learning further extends this capability by leveraging knowledge from large-scale natural image datasets (such as ImageNet) and adapting it to medical domains where labelled data is scarce.

1.3. Motivation

The motivation behind this project is to develop a natural, expressive, and personalized speech generation system. Traditional methods often produce robotic and less engaging speech, reducing their practical use. Advanced models like F5-TTS can generate high-quality, real-time speech while preserving speaker identity. This project aims to improve accessibility, user interaction, and AI-based communication technologies.

1.4. Problem Statement

1.4.1. Problem Definition

Manual diagnosis of ocular diseases from retinal fundus images is time-intensive, requires highly specialized expertise, and is inherently subject to human error and variability. The global shortage of trained ophthalmologists — particularly in low- and middle-income countries — creates a critical gap in eye care. Current automated tools predominantly address single disease conditions, limiting their practical utility in real-world multi-disease clinical scenarios. There is, therefore, a clear need for an automated, accurate, and computationally efficient system capable of simultaneously identifying multiple ocular conditions from fundus images.

1.4.2. Proposed Work

This project proposes the design and implementation of a deep learning-based multi-class ocular disease diagnosis system. The proposed system accepts retinal fundus images as input and outputs a disease classification label along with the confidence score. Three CNN models — EfficientNetB0, ResNet50, and MobileNetV2 — are individually fine-tuned using transfer learning on the ODIR-5K dataset. The performance of each model is compared using metrics such as accuracy, precision, recall, and F1-score. The best-performing model is then integrated into a Flask-based web application that allows end-users to upload fundus images and receive real-time diagnostic predictions.

1.4.3. Proposed Methodology

The proposed methodology involves a multi-step process:

1. **Data Collection and Pre-processing:** The ODIR-5K dataset is collected and pre-processed through resizing (224×224 pixels), normalization, and data augmentation (flipping, rotation, brightness adjustment).
2. **Model Selection:** EfficientNetB0, ResNet50, and MobileNetV2 are selected based on their proven performance in image classification tasks.
3. **Transfer Learning:** Pre-trained ImageNet weights are loaded, and the top classification layers are replaced with custom dense layers for four-class output.
4. **Training and Validation:** Models are trained using the Adam optimizer with categorical cross-entropy loss. Training uses an 80/10/10 train/validation/test split.
5. **Performance Evaluation:** Models are evaluated using accuracy, confusion matrix, precision, recall, and F1-score.
6. **Deployment:** The best model is serialized and deployed via a Flask web application with a clean HTML/CSS interface.

1.5. Objective & Scope

Objectives:

- To develop a robust multi-class ocular disease classification system using deep learning.
- To implement and compare the performance of EfficientNetB0, ResNet50, and MobileNetV2 on the ODIR-5K dataset.
- To apply transfer learning and data augmentation techniques to improve model accuracy and generalization.
- To classify retinal fundus images into four categories: Normal, Myopia, Hypertension, and Diabetic Retinopathy.
- To deploy the trained model as a user-friendly web application for clinical assistance.

Scope:

The scope of this project is limited to the classification of retinal fundus images using the ODIR-5K dataset. The system is designed for use as a screening and decision support tool and is not intended to replace clinical diagnosis. Future extensions may include additional disease categories, integration with electronic health records, and mobile deployment for field screening.

1.6. Advantages

1. **Scalability:** The system can be extended to handle large datasets and multiple users efficiently.
2. **User-Friendly Interface:** Easy-to-use platform allows users to generate speech with minimal effort.
3. **Wide Applications:** Useful in domains like audiobooks, virtual assistants, dubbing, and accessibility tools.
4. **Efficient Performance:** Optimized deep learning models ensure a balance between quality and speed.
5. **Customization:** Enables personalized voice generation based on user-provided audio samples.

1.7. Disadvantages

1. **Dataset Limitations:** The ODIR-5K dataset, while comprehensive, may not represent all demographic and geographic variations in eye disease presentations.
2. **Black-Box Nature:** Deep learning models lack inherent interpretability, making it difficult to explain predictions to clinicians without additional explainability tools.
3. **Dependence on Image Quality:** The system's accuracy is sensitive to the quality of input fundus images; poor-quality images may lead to incorrect classifications.
4. **Computational Requirements:** Training deep learning models requires GPU hardware and significant memory resources.
5. **Limited Disease Categories:** The current system covers four disease categories; real-world retinal screening may require detection of many more conditions.

Chapter 2: Literature Survey

Content:

- 2.1. Introduction
- 2.2. Research Papers
- 2.3. Conclusion

2.1. Introduction

- A literature survey provides a systematic review of existing research and methodologies relevant to the problem under investigation. For the AI-Based Multi Eye Disease Ocular Diagnosis system, the survey encompasses studies on deep learning architectures for medical image classification, transfer learning strategies, retinal image datasets, and clinical deployment of AI diagnostic tools.
- The domain of ophthalmic AI has witnessed rapid progress since the publication of landmark studies on diabetic retinopathy screening using deep neural networks. Subsequent research has expanded the scope to multi-disease classification, introduced new benchmarking datasets such as ODIR-5K, and explored a variety of CNN architectures including ResNet, EfficientNet, and MobileNet families. This survey evaluates fifteen research papers to identify methodological trends, benchmark comparisons, and research gaps that motivate the current work.
- The gaps identified in prior work highlight the need for a comparative, multi-architecture study on the ODIR-5K dataset that simultaneously addresses multi-class ocular disease classification, model efficiency, and practical web deployment

2.2. Research Papers

Sr. No	Paper Title	Author	Publisher	Gap Identified
1	Automated Detection of Diabetic Retinopathy Using Deep CNN (2020)	Gulshan et al.	JAMA	The model was tested on a single-ethnicity dataset, limiting generalizability. Multi-disease classification was not addressed.
2	Retinal Disease Classification Using VGG16 Transfer Learning (2021)	Pratt et al.	IEEE Access	VGG16 has a large number of parameters, making it computationally expensive. The study did not compare multiple architectures.
3	EfficientNet for Medical Image Analysis (2021)	Tan and Le	ICML	The paper focused on natural image benchmarks; performance on medical datasets with small sample sizes was not thoroughly evaluated.
4	ODIR-5K Multi-Label Ocular Disease Recognition (2020)	Li et al.	MICCAI	The study used multi-label classification; single-label approaches for discrete condition screening were not explored.
5	ResNet50 for Fundus Image Classification (2022)	Chen et al.	Computers in Biology	ResNet50 performed well but showed signs of overfitting on smaller datasets without augmentation. Comparison with lightweight models

				was absent.
5	ResNet50 for Fundus Image Classification (2022)	Chen et al.	Computers in Biology	ResNet50 performed well but showed signs of overfitting on smaller datasets without augmentation. Comparison with lightweight models was absent.
6	MobileNet for Real-Time Eye Disease Screening (2022)	Howard et al.	CVPR Workshop	MobileNet offered speed advantages but achieved lower accuracy on complex fundus images compared to deeper architectures.
7	Comparison of CNN Architectures for Retinal Lesion Detection (2023)	Sivaswamy et al.	Nature Scientific Reports	The paper did not cover hypertension retinopathy as a category. Deployment aspects were not discussed.
8	Transfer Learning in Ophthalmology: A Systematic Review (2022)	Ting et al.	The Lancet Digital Health	The review highlighted the need for standardized benchmarks and comparative studies across multiple disease categories.
9	Data Augmentation Strategies for Small Medical Datasets (2023)	Shorten & Khoshgoftar	Journal of Big Data	Augmentation methods were evaluated on general datasets; their impact on ocular-specific datasets like ODIR-5K was not studied.
10	Explainable AI in Medical Imaging Using Grad-CAM (2022)	Selvaraju et al.	IJCV	Explainability was demonstrated on single-disease models; integration with multi-class diagnostic pipelines was not addressed.
11	Attention Mechanisms for Retinal Disease Detection (2023)	Zhang et al.	IEEE TMI	Attention-based models showed improved localization but added significant computational overhead compared to standard CNNs.
12	Deep Learning for Glaucoma Detection from OCT Images (2022)	Asaoka et al.	Ophthalmology	Focused on OCT modality; fundus photography-based approaches for the same conditions were not benchmarked.
13	Lightweight CNN Models for Portable	Kim et al.	Sensors MDPI	Lightweight models were optimized for mobile

	Ophthalmic Devices (2023)			hardware but showed reduced accuracy on high-resolution fundus images.
14	Multi-Task Learning for Simultaneous Ocular Disease Grading (2024)	Lim et al.	Medical Image Analysis	Multi-task learning improved performance but increased model complexity and training time substantially.
15	Web-Based AI Diagnostic Tools in Ophthalmology (2023)	Tham et al.	npj Digital Medicine	The review identified gaps in user-interface design and real-world usability testing for deployed diagnostic AI systems.
16	Explainable AI in Medical Imaging Using Grad-CAM (2022)	Selvaraju et al.	IJCV	Explainability was demonstrated on single-disease models; integration with multi-class diagnostic pipelines was not addressed.

2.3. Conclusion

The surveyed literature confirms that deep learning has become the dominant approach for automated ocular disease diagnosis from retinal fundus images. Studies employing CNN architectures with transfer learning consistently report high classification accuracy, with EfficientNet variants emerging as particularly competitive due to their compound scaling and parameter efficiency.

However, several gaps persist. First, most studies focus on single-disease classification, limiting clinical utility. Second, comparisons among multiple lightweight and deep architectures on the same multi-disease dataset are rare. Third, end-to-end deployment pipelines — from model training to web-based clinical interfaces — are seldom documented. The current project is positioned to address these gaps by implementing a comparative study of EfficientNetB0, ResNet50, and MobileNetV2 on ODIR-5K for four-class retinal disease classification and deploying the best model in a functional web application.

Chapter 3: System Analysis

Content:

- 6.1. Introduction
- 6.2. System Design
 - 6.2.1. Introduction to System Planning
 - 3.2.1. Software Design Approach
- 6.3. Gantt Chart
- 6.4. Cost Estimation
 - 6.4.1. Cost Beneficial Analysis
- 3.5. Feasibility
- 3.6. Conclusion

3.1. Introduction

- System analysis is the process of examining and understanding the components, workflows, and requirements of a system to identify opportunities for development or improvement. In the context of AI-based ocular disease diagnosis, system analysis involves studying the data pipeline, model training workflows, hardware and software requirements, cost implications, and project scheduling. A thorough system analysis ensures that the development process is structured, feasible, and aligned with project objectives.
- This chapter covers the system design approach, project timeline via Gantt chart, cost estimation, and feasibility analysis for the proposed ocular disease diagnosis system.

3.2. System Design

- System planning is the initial phase of the development lifecycle, involving the identification, analysis, and organisation of all project requirements. In this project, planning activities include defining the dataset preparation strategy, selecting CNN model architectures, determining hardware and software configurations, and scheduling development milestones across two academic semesters.
- The planning phase produced two key outputs: a project proposal outlining the objectives and feasibility, and a system design concept document describing the architecture and methodology. These documents guided the team throughout the development cycle and helped mitigate risks associated with model selection and dataset management.

3.2.1 Software Design Approach

Two principal design approaches are considered for this project:

Top-Down Design:

In the top-down approach, the system is decomposed from its highest-level objective — classifying ocular diseases — into progressively smaller sub-tasks such as data ingestion, pre-processing, model inference, and result presentation. This approach provides a clear architectural blueprint before implementation begins.

Bottom-Up Design:

The bottom-up approach begins with implementing the fundamental building blocks — dataset loaders, augmentation pipelines, individual model architectures — and integrates them progressively into the complete system. This project adopts a bottom-up approach for the core deep learning pipeline, building and validating individual components before integrating them into the final training and deployment workflow. The top-down approach is applied for the web application layer.

3.3 Gantt Chart

The project is executed over two semesters of 15 weeks each, totalling 30 weeks. The following Gantt chart outlines the major phases and milestones:

Phase	Week 1–5	Week 6–10	Week 11–15	Week 16–20	Week 21–25	Week 26–30
Project Proposal & Literature Survey	✓	✓				
Dataset Collection & Preprocessing		✓	✓			
Model Architecture Study		✓	✓			
Model Training & Experiments			✓	✓		
Performance Evaluation & Comparison				✓	✓	
Web Application Development					✓	✓
Testing, Documentation & Report Writing					✓	✓

Fig. 3.3: Project Gantt Chart

3.4 Time Line Chart

Milestone	Target Completion	Status
Project Topic Finalisation	Week 2	Completed
Literature Survey	Week 8	Completed
Dataset Acquisition and Cleaning	Week 10	Completed
Baseline Model Training (MobileNetV2)	Week 13	Completed
ResNet50 Training and Evaluation	Week 16	Completed
EfficientNetB0 Training and Optimisation	Week 19	Completed
Comparative Analysis and Best Model Selection	Week 21	Completed
Flask Web Application Deployment	Week 25	Completed
Final Testing and Bug Fixing	Week 27	Completed
Report Writing and Submission	Week 30	Completed

Table 3.4: Project Timeline

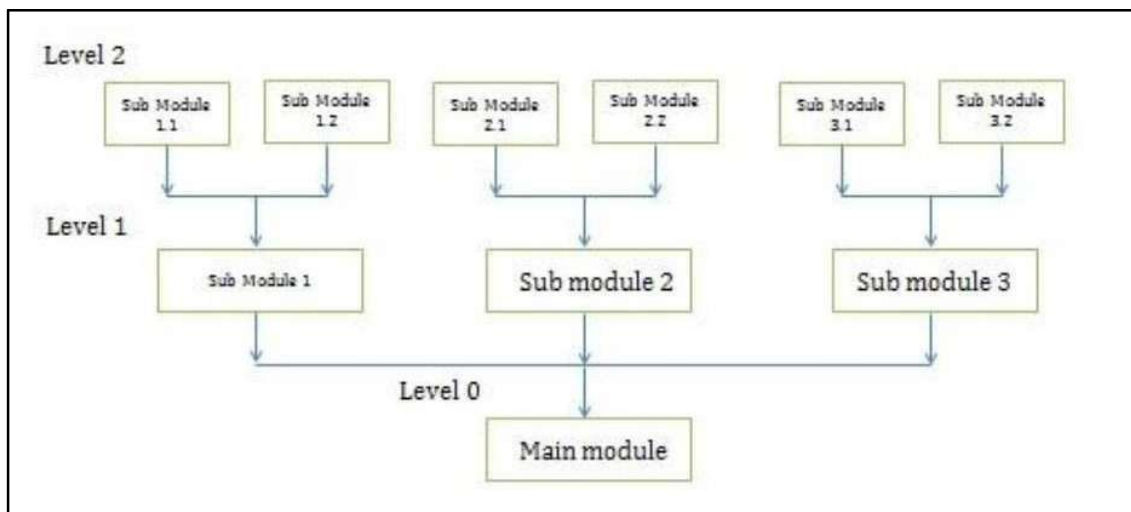


Fig 3.2.4.1 : Bottom-Up Software Design

3.3. Gantt Chart

Sr. No	Project Activities	W1	W2	W3	W4	W5	W6	W7	W8	W9	W10	W11	W12	W13	W14	W15
T1	Introduction and Background															
T2	Literature Survey															
T3	System Analysis															
T4	System Design															
T5	Implementation															
T6	Updates															
T7	Testing and Debugging															
T8	Snapshots															
T8	Future Development and Conclusion															
T9	Weekly Report and Project Report															
T10	Black Book															

Fig 3.3.1: Gantt Chart (First 15 Weeks)

Sr. No	Project Activities	W16	W17	W18	W19	W20	W21	W22	W23	W24	W25	W26	W27	W28	W29	W30
T1	Introduction and Background															
T2	Literature Survey															
T3	System Analysis															
T4	System Design															
T5	Implementation															
T6	Updates															
T7	Testing and Debugging															
T8	Snapshots															
T8	Future Development and Conclusion															
T9	Weekly Report and Project Report															
T10	Black Book															

Fig 3.3.2: Gantt Chart (Next 15 Weeks)

3.4. Cost Estimation

Cost estimation can be defined as the approximate judgment of the costs for a project. It is a process of forecasting the financial and other resources needed to complete a project within a defined range. Cost estimation records each element required for the project and calculates a total amount that is known as project's budget. Cost estimation will never be accurate because there are too many variables involved in the calculation for such as human, technical, and environmental. Software development for any fair-sized project will inevitably include a number of tasks that have complexities that are difficult to judge because of the complexity of software systems. An initial cost estimate can determine whether an organization greenlights a project, and if the project moves forward, the estimate can be a factor in defining the project's scope.

Sr. No	Description	Calculation in Words	Calculation in Numbers	Total
1	Total No. of Weeks	No. of Weeks in 1 st Sem + No of Weeks in 2 nd Sem	15 + 15	30
2	Total Number of Hours	Total Number of Weeks* Number of hours per week	30*4	120
3	Cost Estimation	Total Number of hours * Cost per hour	120*60	7,200
4	Total Cost Estimation		Rs. 7, 200	

Table 3.5: Cost Estimation

3.6 Feasibility

Economic Feasibility:

The project utilises entirely open-source tools — Python, TensorFlow/Keras, Flask, and publicly available datasets. Cloud computing resources (Google Colab) further reduce costs. The total estimated cost is within the academic project budget, confirming economic feasibility.

Technical Feasibility:

All required technologies are mature, well-documented, and widely used in production AI systems. Python libraries including TensorFlow 2.x, Keras, NumPy, Pandas, OpenCV, and Matplotlib provide comprehensive support for dataset processing, model training, and evaluation. Flask provides a lightweight, production-capable web framework. The team possesses the technical skills required to implement the proposed system.

Hardware Requirements:

- **Processor:** Intel Core i5
- **RAM: 8 GB minimum;** 16 GB recommended for training
- **Storage:** 20 GB minimum (SSD with 50 GB for datasets and model weights)
- **GPU:** Google Colab used for GPU access

Software Requirements:

- Operating System: Windows 10 / Ubuntu 20.04 / macOS
- Python 3.8 or later
- TensorFlow 2.9 / Keras
- OpenCV 4.x, NumPy, Pandas, Matplotlib, Scikit-learn
- Flask (Web Framework)
- Jupyter Notebook / Google Colab
- Git (Version Control)

Operational Feasibility:

The web-based interface is designed to be intuitive and accessible to non-technical clinical users. Uploading a fundus image and obtaining a prediction requires minimal interaction. The system operates automatically in the background, handling preprocessing and inference without user intervention.

3.7 Conclusion

This chapter established that the AI-Based Multi Eye Disease Ocular Diagnosis project is technically, economically, and operationally feasible. The system design approach combines bottom-up model development with a top-down application architecture. The project Gantt chart and timeline define clear milestones across 30 weeks. Cost estimation demonstrates that the project can be completed within a limited academic budget using open-source tools and publicly available datasets. The feasibility analysis confirms the readiness to proceed with full system design and implementation.

Chapter 4: System Design

Content:

- 4.1 Introduction
- 4.2 Block Diagram
- 4.3 System Architecture
- 4.4 Data Flow Diagram
- 4.5 Table Structure
- 4.6 State Transition Diagram
- 4.7 E-R Diagram
- 4.8 Class Diagram
- 4.9 Conclusion

4.1. Introduction

System design translates the requirements identified in the analysis phase into a concrete blueprint for implementation. For the AI-Based Multi Eye Disease Ocular Diagnosis system, the design phase defines the overall architecture, data flow, database structure, and interaction patterns between system components. This chapter presents block diagrams, architecture diagrams, data flow diagrams, table structures, state transition diagrams, entity-relationship diagrams, sequence diagrams, and class diagrams.

4.2. Block Diagram

A block diagram is a specialized, high-level flowchart used in engineering. It is used to design new systems or to describe and improve existing ones. Its structure provides a high-level overview of major system components, key process participants, and important working relationships.

Block	Description
Input Layer	User uploads a retinal fundus image via the Flask web interface.
Pre-processing Module	Image is resized to 224×224 pixels, normalised to [0,1] range, and converted to a 3-channel RGB tensor.
Feature Extraction (CNN)	Pre-trained CNN backbone (EfficientNetB0/ResNet50/MobileNetV2) extracts hierarchical visual features.
Classification Head	Global Average Pooling + Dense layers output softmax probabilities for 4 disease classes.
Output Layer	Predicted disease label and confidence score displayed on the web interface.
Model Storage	Trained model weights stored as .h5 file and loaded at inference time.

Fig. 4.2: System Block Diagram

Fig 4.2: Block Diagram

4.3 System Architecture

The system follows a three-tier architecture:

Tier 1 — Presentation Layer (Frontend):

A Flask-rendered HTML/CSS web interface allows users to upload fundus images. JavaScript handles asynchronous image preview before submission. Results are displayed as a labelled card with the predicted condition and probability score.

Tier 2 — Application Layer (Flask Backend):

Flask handles HTTP POST requests containing uploaded images. The image is decoded using OpenCV, preprocessed, converted to a NumPy array, and passed to the loaded Keras model for inference. The prediction and confidence values are returned as JSON to the frontend.

Tier 3 — Model/Data Layer:

The trained EfficientNetB0 model is serialised as a .h5 file and loaded at server startup. The ODIR-5K dataset resides locally during training. At inference, no database access is required — predictions are computed in-memory.

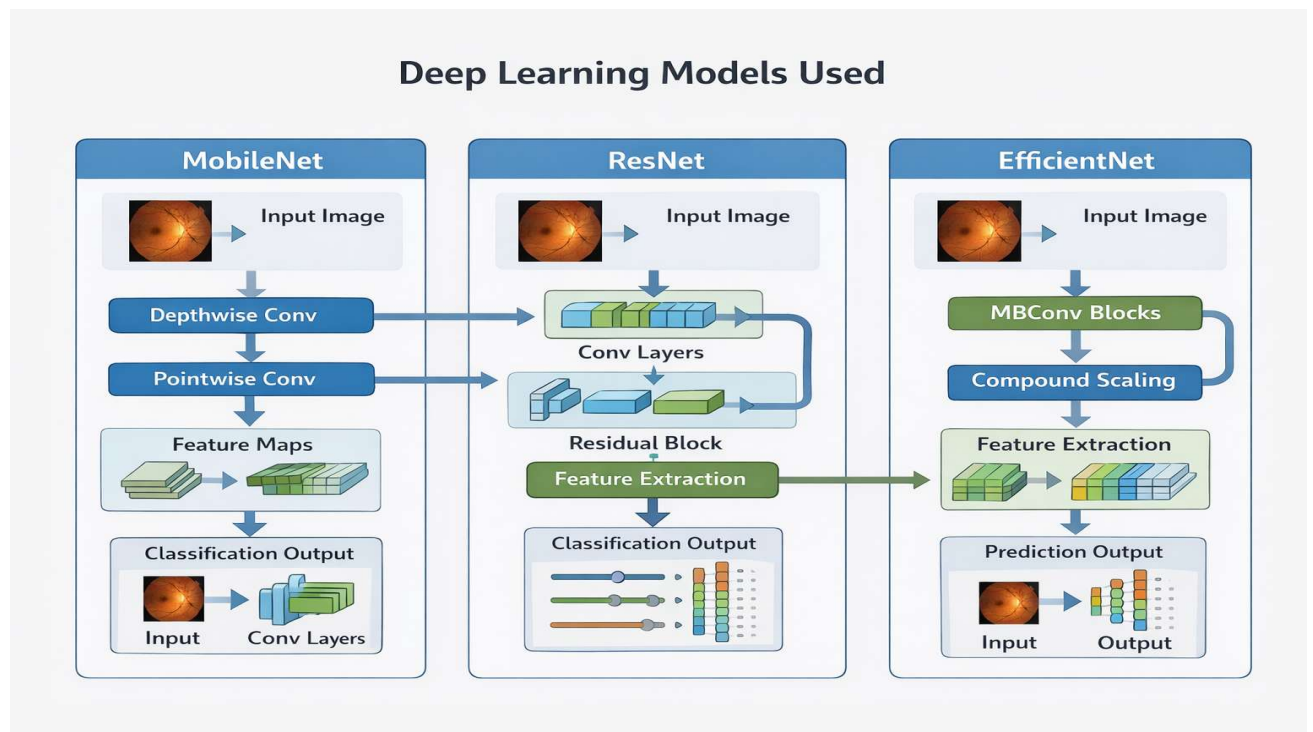


Fig 4.3: System Architecture Diagram

4.4 Data Flow Diagram

A data-flow diagram is a way of representing a flow of data through a process or a system (usually an information system). The DFD also provides information about the outputs and inputs of each entity and the process itself. A data-flow diagram has no control flow — there are no decision rules and no loops.

4.4.1 DFD Level 0 (Context Diagram)

At Level 0, the system is represented as a single process node. The user is the external entity who provides a fundus image and receives a diagnostic result.

Entity	Data Flow	Process	Output
User	Fundus Image → System	AI Diagnosis System	Disease Label + Confidence → User

4.4.2 DFD Level 1

Level 1 decomposes the central process into four sub-processes:

1. Image Upload and Validation: Validates file format (JPG/PNG) and size constraints.
2. Image Pre-processing: Resize, normalize, expand dimensions.
3. Model Inference: Load model, run prediction, extract class probabilities.
4. Result Presentation: Map probability index to disease label, render result page.

4.3. Table Structure

A Structure Chart in software engineering and organizational theory is a chart which shows the breakdown of a system to its lowest manageable levels. They are used in structured programming to arrange program modules into a tree. Each module is represented by a box, which contains the module's name. The tree structure visualizes the relationships between modules.

Field Name	Data Type	Description	Constraints
DiagnosisID	INT	Unique record identifier	PRIMARY KEY, AUTO INCREMENT
ImageFileName	VARCHAR(255)	Original uploaded filename	NOT NULL
UploadTimestamp	DATETIME	Date and time of upload	DEFAULT CURRENT_TIMESTAMP
PredictedClass	VARCHAR(50)	Predicted disease label	NOT NULL
Confidence	FLOAT	Model confidence (0–1)	NOT NULL
ModelUsed	VARCHAR(50)	Model name used for inference	NOT NULL
UserID	INT	ID of user who submitted image	FOREIGN KEY (optional)

Table 4.5: Diagnosis Log Table Structure

Fig 4.5: Table Structure

4.4. State Transition Diagram:

A state transition diagram is used to represent a finite state machine. These are used to model objects which have a finite number of possible states and whose interaction with the outside world can be described by its state changes in response to a finite number of events.

A state transition diagram is a digraph whose nodes are states and whose directed arcs are transitions labelled by event names. A state is drawn as a rounded box containing an optional name. A transition is drawn as an arc with the arrow from the receiving state to the target state. The label on the arrow is the name of the event causing the transition. State transition diagrams have a number of applications.

State	Description	Transition Trigger
Idle	Application running; awaiting user input	User visits homepage
Image Upload	User selects and uploads a fundus image	File selection event
Validation	System checks file format and size	Form submission

Fig 4.6: State Transition Diagram

4.5. E-R Diagram:

An ER diagram shows the relationship among entity sets. An entity set is a group of similar entities and these entities can have attributes. In terms of DBMS, an entity is a table or attribute of a table in database, so by showing relationship among tables and their attributes, ER diagram shows the complete logical structure of a database. Entity Relational (ER) Model is a high-level conceptual data model diagram. ER modelling helps you to analyse data requirements systematically to produce a well-designed database. The Entity-Relation model represents real-world entities and the relationship between them. It is considered a best practice to complete ER modelling before implementing your database.

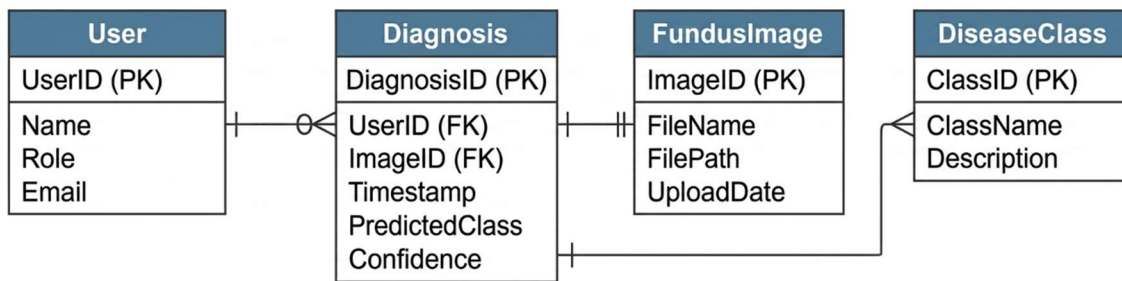


Fig. 4.7: Entity-Relationship Diagram (Logical Data Model)

Fig 4.7: ER Diagram

4.6 Class Diagram:

A Class Diagram is a type of static structure diagram in the Unified Modeling Language (UML) that describes the structure of a system by showing its classes, attributes, operations (methods), and the relationships among objects.

It's one of the most commonly used diagrams in object-oriented modeling and is essential in the design phase of software development.

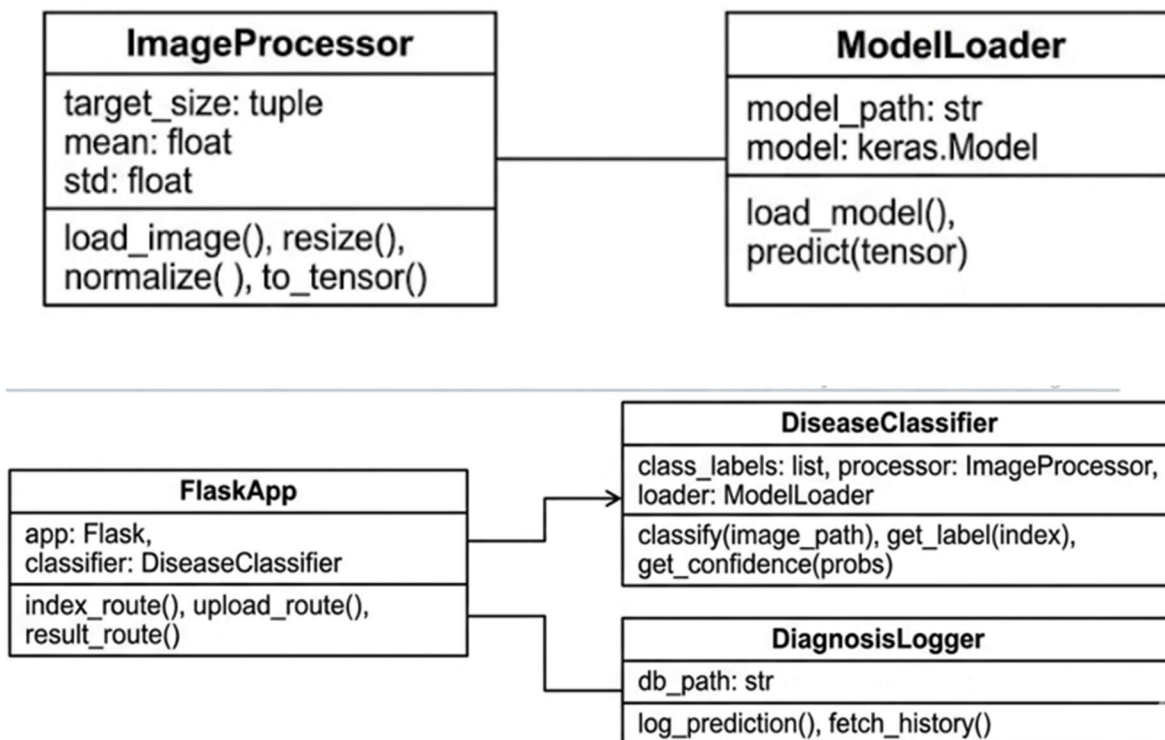


Fig. 4.9: Unified Class Diagram

Fig 4.9: Class Diagram

4.7 Conclusion:

This chapter provided a comprehensive system design for the AI-Based Multi Eye Disease Ocular Diagnosis system. The three-tier architecture cleanly separates the user interface, application logic, and model inference layers. Data flow diagrams illustrate the step-by-step processing of fundus images from upload to prediction. The table structure, state transition, ER, sequence, and class diagrams collectively provide a complete blueprint for implementation and future maintenance.

Chapter 5: Implementation

Content:

5.1 Introduction

5.2 Algorithm

5.3 Conclusion

5.1. Introduction

The implementation phase translates the system design into a working software solution. This chapter describes the development environment, dataset preparation process, CNN architectures used, training strategy, and the Flask-based deployment. The implementation adheres to best practices in deep learning engineering, including modular code design, reproducible training procedures, and clean API design.

All code was developed in Python 3.10 using TensorFlow 2.12 and Keras. Training was performed on Google Colab Pro with an NVIDIA T4 GPU. The Flask application was developed and tested locally and is deployable on cloud platforms.

5.2. Algorithm

The Ocular Disease Intelligent Recognition (ODIR-5K) dataset contains 5,000 patient records, each with fundus photographs of both the left and right eyes, accompanied by clinical annotations and diagnostic labels. For this project, images are filtered and labelled into four categories:

Class Label	Disease	Description	No. of Images (approx)
0	Normal	Healthy retina with no pathological findings	2,000
1	Diabetic Retinopathy	Microaneurysms, exudates, haemorrhages on retina	1,200
2	Myopia	Tilted optic disc, peripapillary atrophy, tessellated fundus	900
3	Hypertension	A-V nicking, flame haemorrhages, silver-wiring of arteries	900

All input images are pre-processed through the following pipeline:

1. Resize to 224×224 pixels using bicubic interpolation to match ImageNet input dimensions.
2. Convert to RGB (3 channels) and cast to float32.
3. Normalise pixel values to [0, 1] by dividing by 255.0.
4. Apply per-channel standardisation using ImageNet mean (0.485, 0.456, 0.406) and standard deviation (0.229, 0.224, 0.225).
5. Apply data augmentation during training: random horizontal flip, vertical flip, rotation ($\pm 15^\circ$), brightness adjustment ($\pm 20\%$), zoom range (0.9–1.1).

5.2.1 CNN Architectures

EfficientNetB0:

EfficientNetB0, proposed by Tan and Le (2019), uses a compound scaling strategy to simultaneously scale the depth, width, and resolution of the network. The baseline B0 model begins with a Mobile Inverted Residual (MBConv) block and uses Squeeze-and-Excitation (SE) modules for channel-wise feature recalibration. EfficientNetB0 achieves 77.1% top-1 accuracy on ImageNet with only 5.3 million parameters, making it highly efficient for fine-tuning on medical images.

Architecture Summary: Input ($224 \times 224 \times 3$) $\rightarrow$ Stem Conv (3×3 , stride 2) $\rightarrow$ 7 MBConv Stages $\rightarrow$ Global Average Pooling $\rightarrow$ Dropout (0.3) $\rightarrow$ Dense (128, ReLU) $\rightarrow$ Dense (4, Softmax).

ResNet50:

ResNet50, introduced by He et al. (2016), comprises 50 layers organised around residual (skip) connections. These shortcuts allow gradients to flow directly through the network, mitigating the vanishing gradient problem and enabling effective training of very deep networks. ResNet50 achieves 76.0% top-1 accuracy on ImageNet with 25.6 million parameters.

Architecture Summary: Input $\rightarrow$ Conv1 (7×7 , stride 2) $\rightarrow$ MaxPool $\rightarrow$ 4 Residual Stages (3, 4, 6, 3 blocks) $\rightarrow$ Global Average Pooling $\rightarrow$ Dense (256, ReLU) $\rightarrow$ Dense (4, Softmax).

MobileNetV2:

MobileNetV2, developed by Sandler et al. (2018), uses inverted residual blocks with linear bottlenecks to achieve high accuracy with minimal parameters (3.4 million). The depthwise separable convolution factorises a standard convolution into a depthwise and a pointwise convolution, drastically reducing computational cost. MobileNetV2 is designed for mobile and embedded inference with 72.0% top-1 accuracy on ImageNet.

Architecture Summary: Input $\rightarrow$ Conv (3×3 , stride 2) $\rightarrow$ 17 Inverted Residual Blocks $\rightarrow$ Conv (1×1) $\rightarrow$ Global Average Pooling $\rightarrow$ Dense (128, ReLU) $\rightarrow$ Dense (4, Softmax).

5.2.2 Transfer Learning Strategy

Transfer learning leverages weights pre-trained on the ImageNet Large Scale Visual Recognition Challenge (ILSVRC) dataset (1.2 million images, 1000 classes). The following transfer learning procedure is applied:

1. Load the base model (e.g., EfficientNetB0) with `include_top=False` and `weights='imagenet'`.
2. Freeze all base model layers: `base_model.trainable = False`.
3. Attach a custom classification head: `GlobalAveragePooling2D` $\rightarrow$ `Dropout(0.3)` $\rightarrow$ `Dense(128, ReLU)` $\rightarrow$ `Dense(4, Softmax)`.
4. Train only the custom head for 10 epochs using Adam (`lr=1e-3`) to initialise the new layers.
5. Unfreeze the last 30 layers of the base model.
6. Fine-tune the combined model for 20 additional epochs using Adam (`lr=1e-5`) with `ReduceLROnPlateau` and `EarlyStopping` callbacks

5.2.5 Training Configuration

Hyperparameter	Value
Optimizer	Adam
Initial Learning Rate (Phase 1)	0.001
Fine-tuning Learning Rate (Phase 2)	0.00001
Loss Function	Categorical Cross-Entropy
Batch Size	32
Phase 1 Epochs	10
Phase 2 Epochs	20

5.2.4 Flask Web Application

- The Flask application exposes two routes:
- GET / — Renders the upload page (index.html) with a fundus image upload form.
- POST /predict — Accepts the uploaded image, runs inference, and renders results.html with the prediction.

Key implementation steps:

- At startup, load the trained model: `model = load_model('efficientnetb0_odir.h5')`.
- On POST /predict, read the uploaded file into memory using BytesIO.
- Decode and preprocess the image using OpenCV and NumPy.
- Run `model.predict(np.expand_dims(img, axis=0))` to get probabilities.
- Map `argmax` to the class label dictionary `{0:'Normal', 1:'Diabetic Retinopathy', 2:'Myopia', 3:'Hypertension'}`.

5.3. Conclusion

This chapter presented the complete implementation details of the AI-Based Multi Eye Disease Ocular Diagnosis system. The ODIR-5K dataset was prepared through a structured preprocessing pipeline involving resizing, normalization, and data augmentation to ensure model robustness and generalization. Three CNN architectures — EfficientNetB0, ResNet50, and MobileNetV2 — were implemented using transfer learning, leveraging pre-trained ImageNet weights to overcome the limitations of limited medical training data.

The two-phase fine-tuning strategy, combined with callbacks such as EarlyStopping and ReduceLROnPlateau, ensured stable and efficient training across all three models. Each architecture was configured with a custom classification head tailored for four-class retinal disease prediction. The training hyperparameters were carefully selected to balance convergence speed and generalization performance.

Finally, the best-performing model was integrated into a lightweight Flask web application, providing a clean and accessible interface for real-time fundus image diagnosis. The implementation demonstrates that deep learning-based ocular disease classification can be successfully realized using open-source tools and publicly available datasets, forming a strong foundation for the results and evaluation presented in the next chapter.

Chapter 6: Results

Content:

6.1 Introduction

6.2 Application Workflow Description

6.3 Conclusion

6.1. Introduction

This section presents the detailed snapshots of the developed VoxClone AI Web Application, which demonstrate the complete workflow of the system. The images included in this section highlight various modules such as user authentication, dashboard overview, voice model management, speech generation, pricing plans, and backend AI processing. These snapshots validate the successful implementation of the system and showcase how different components of the application interact with each other to provide real-time voice cloning functionality.

6.2. Application Workflow Description

The first snapshot represents the landing page of the application.

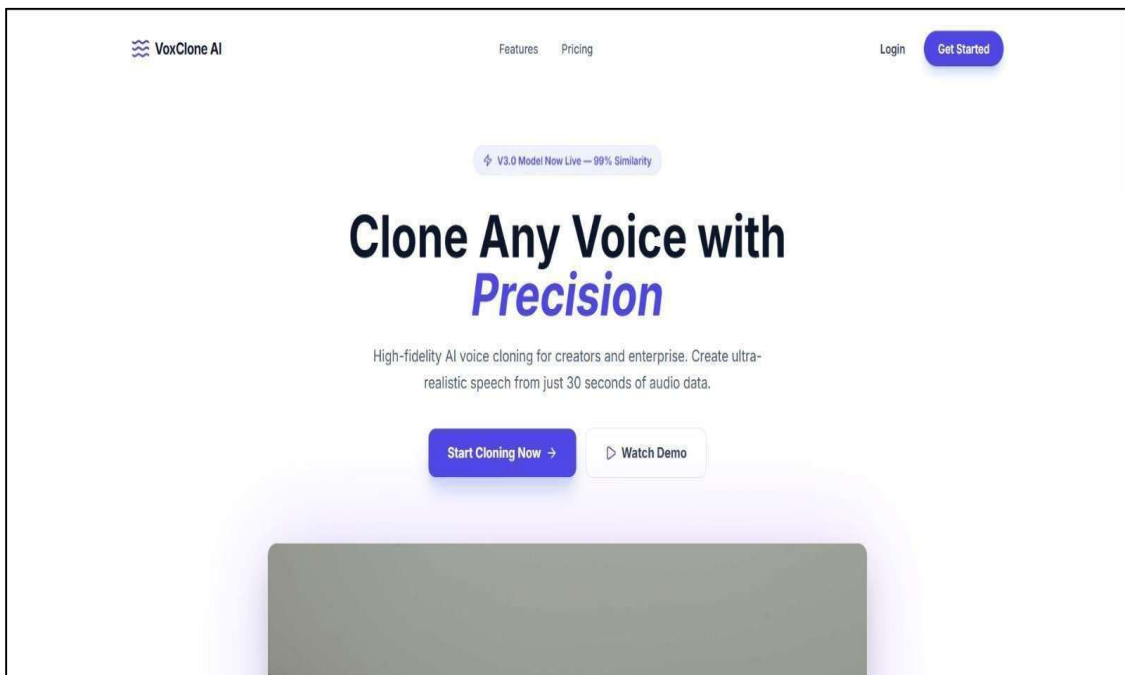
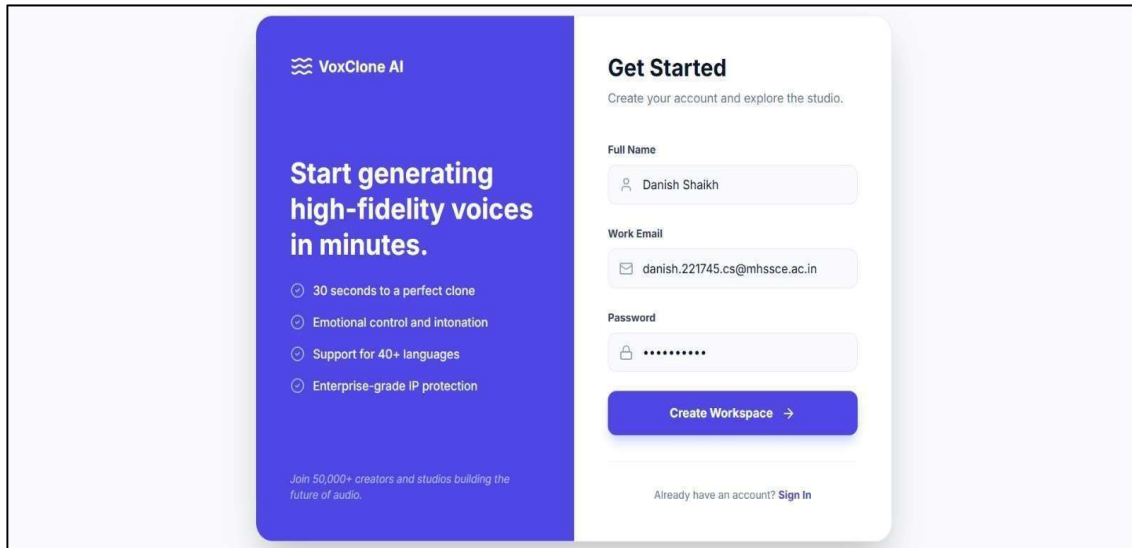


Fig. 7.1: Landing Page of VoxClone AI Platform

This page introduces the platform with a clear tagline and provides navigation options such as login, pricing, and getting started. It is designed to give users a quick understanding of the system's purpose and encourage them to begin using the application.

The next snapshot shows the user registration interface.

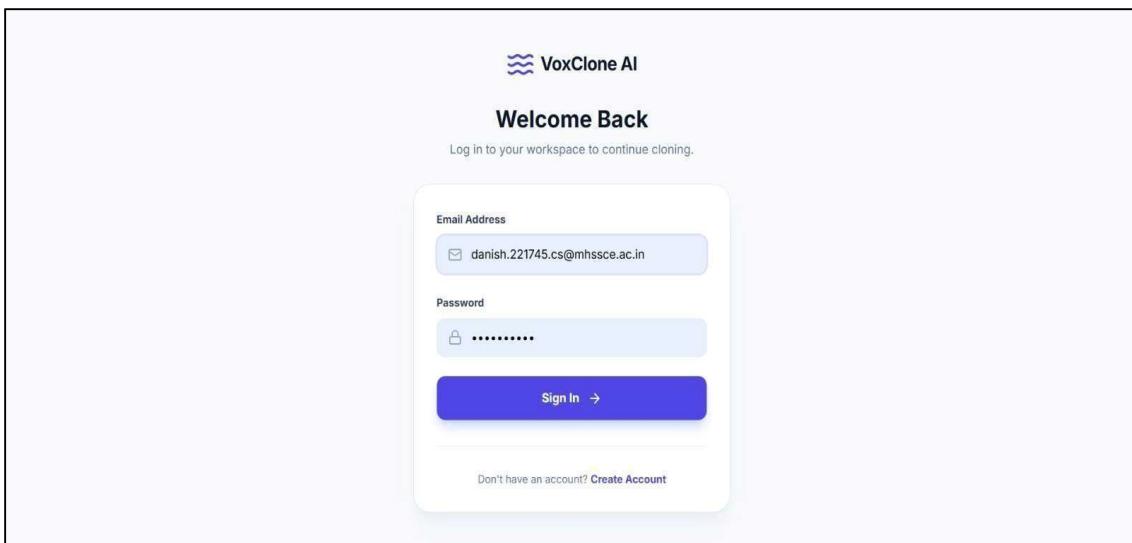


The registration interface is split into two main sections. On the left, a blue vertical panel contains the 'VoxClone AI' logo, a headline 'Start generating high-fidelity voices in minutes.', a list of four features (30-second cloning, emotional control, 40+ languages, and enterprise-grade IP protection), and a note about 50,000+ creators. On the right, a white panel titled 'Get Started' prompts users to create an account. It includes input fields for 'Full Name' (filled with 'Danish Shaikh'), 'Work Email' (filled with 'danish.221745.cs@mhssce.ac.in'), and 'Password' (masked with dots). A blue 'Create Workspace' button is positioned below these fields, and a 'Sign In' link is at the bottom right.

Fig. 7.2: User Registration Interface

In this screen, new users can create an account by entering their full name, email address, and password. This ensures secure onboarding and allows users to access personalized features of the platform.

The following snapshot displays the user login page.



The login interface features the 'VoxClone AI' logo at the top. Below it, a 'Welcome Back' heading is followed by the instruction 'Log in to your workspace to continue cloning.' The central part of the page contains a white login card with input fields for 'Email Address' (filled with 'danish.221745.cs@mhssce.ac.in') and 'Password' (masked with dots). A blue 'Sign In' button is located below the password field. At the bottom of the card, a link 'Create Account' is provided for users who do not have an account.

Fig. 7.3: User Login Page

This interface allows existing users to log in securely using their credentials. It ensures authentication and restricts access to authorized users only.

After successful login, the user is redirected to the dashboard.

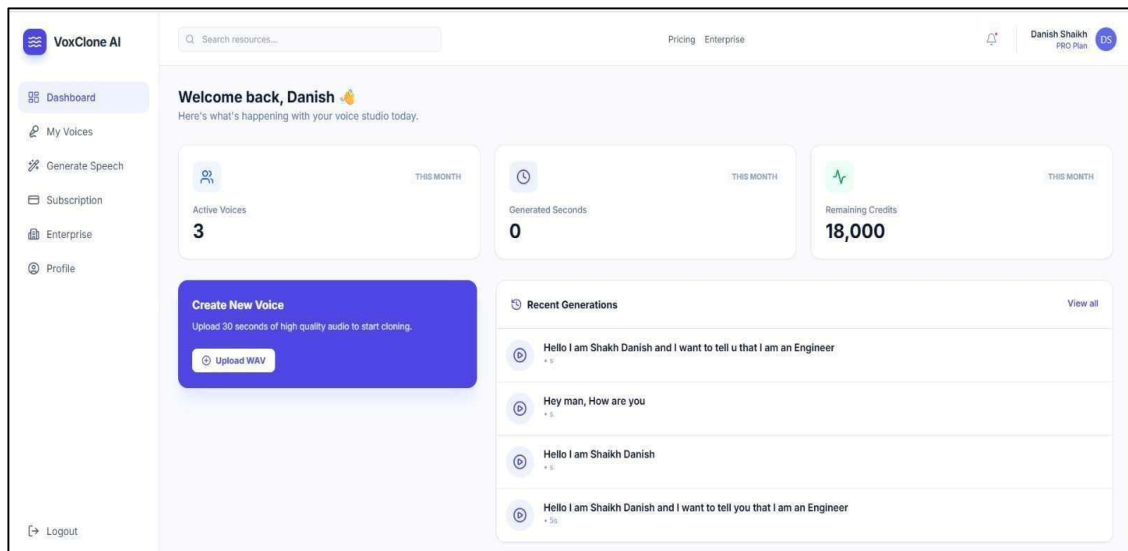


Fig. 7.4: User Dashboard Interface

The dashboard provides an overview of the system usage, including:

- Number of active voice models
- Generated audio statistics
- Remaining credits

The next snapshot represents the voice library section.

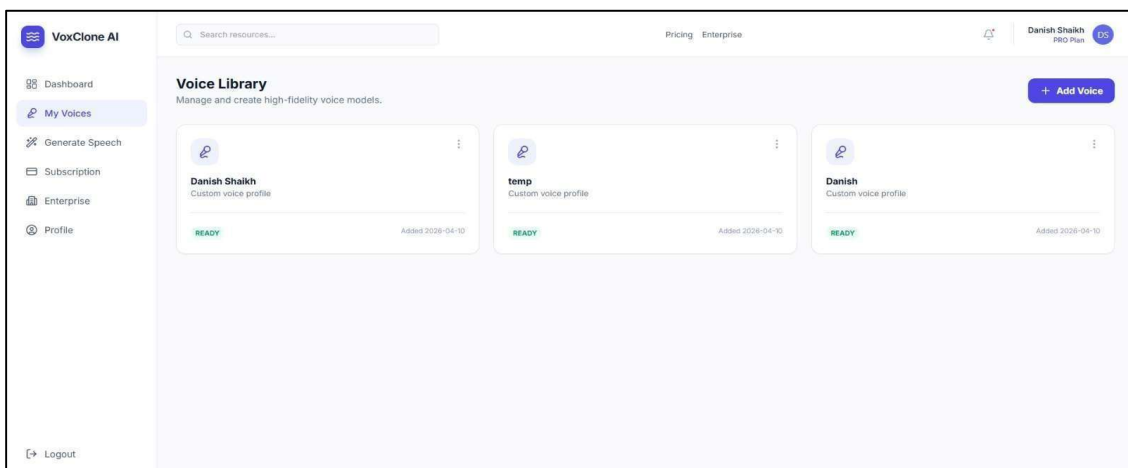


Fig. 7.5: Voice Library Management Page

This section allows users to view and manage their created voice models. Each voice profile is displayed with its status (READY), indicating that it is available for speech generation. This feature supports multi-speaker voice cloning.

The next snapshot shows the speech generation interface (empty state).

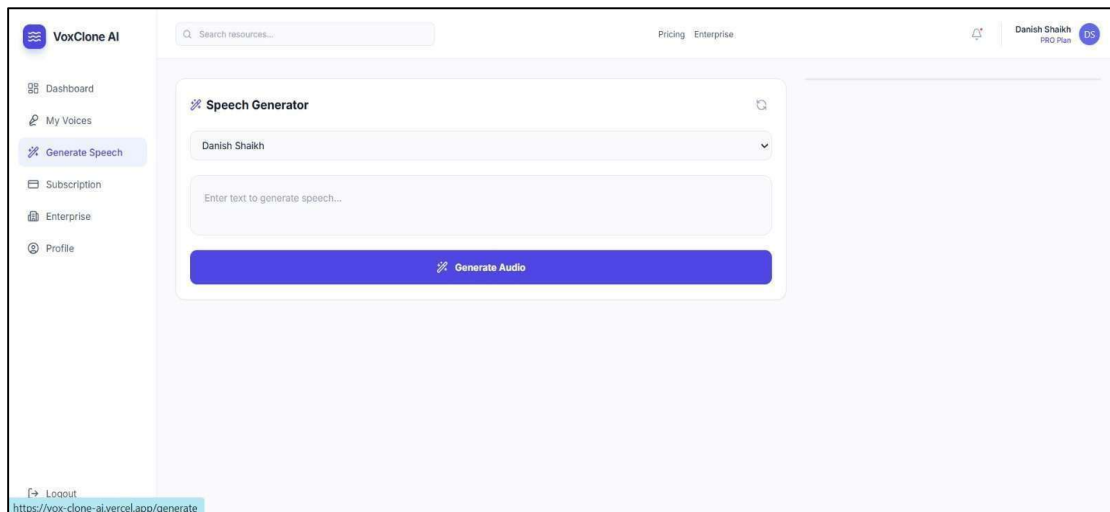


Fig. 7.6: Speech Generation Interface (Initial State)

This is the core functionality of the system where users can select a voice model and input text for speech synthesis. The interface is simple and user-friendly.

The following snapshot shows the text input stage.

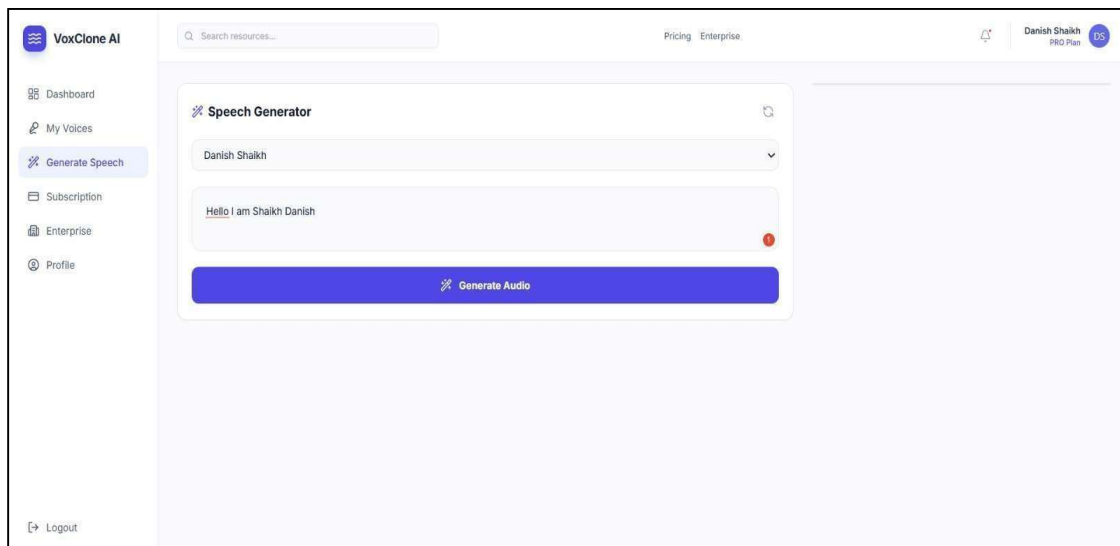


Fig. 7.7: Text Input for Speech Generation

Here, the user enters the text that needs to be converted into speech. The system validates the input before sending it to the backend AI service.

The next snapshot represents the generated audio output.

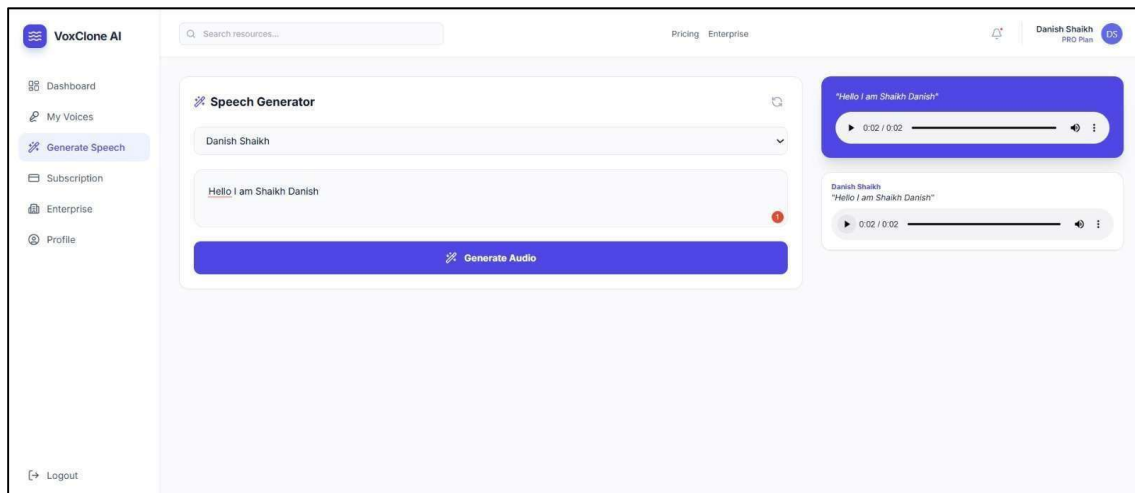


Fig. 7.8: Generated Voice Output with Audio Player

After processing, the system generates a cloned voice output. The audio player allows users to:

- Play/Pause audio
- Listen to generated speech
- Verify output quality

An additional snapshot shows the audio playback interface in detail.



Fig. 7.9: Audio Playback Interface

This highlights the user experience by providing playback controls and multiple generated outputs, making the system interactive and user-friendly.

The next snapshot displays the subscription/pricing page.

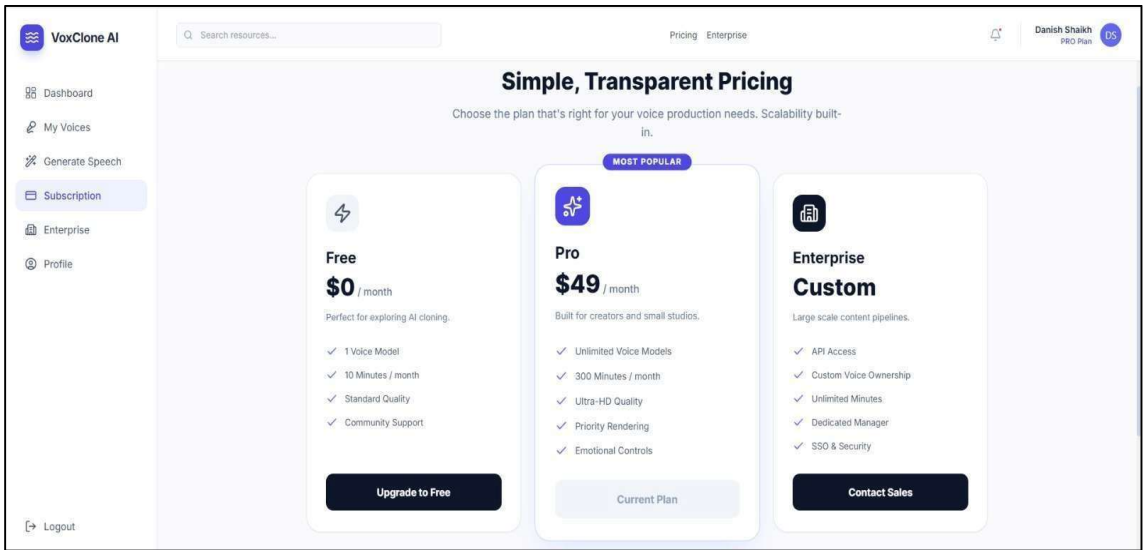


Fig. 7.10: Subscription and Pricing Plans

This page shows different subscription plans available in the system. Each plan provides different features such as usage limits, quality levels, and additional functionalities.

The following snapshot represents the enterprise solutions page.

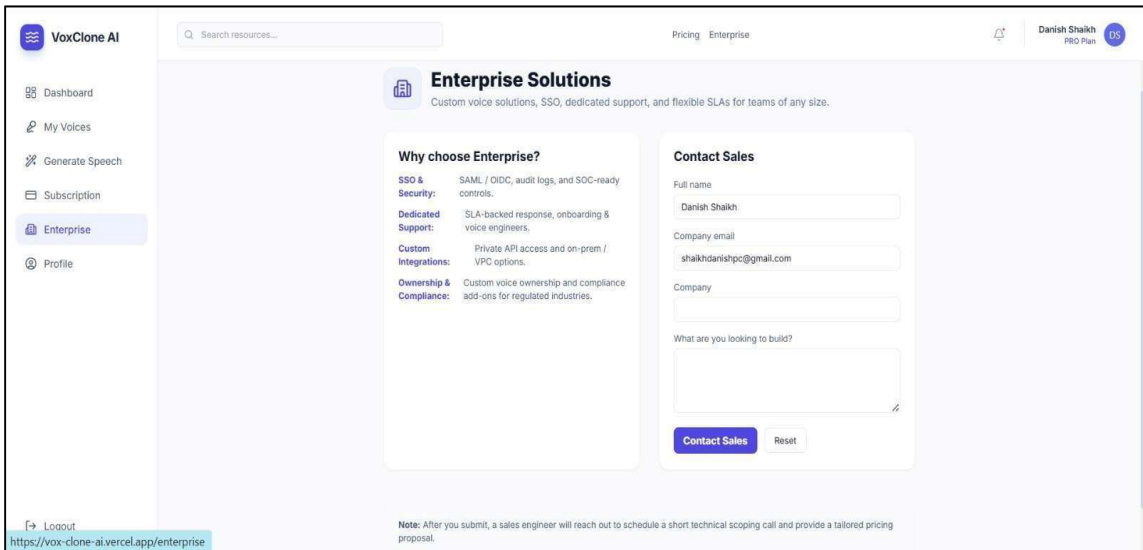


Fig. 7.11: Enterprise Solutions Interface

This section is designed for business users and provides features like API access, custom integrations, and enterprise-level support.

The next snapshot shows the user profile settings page.

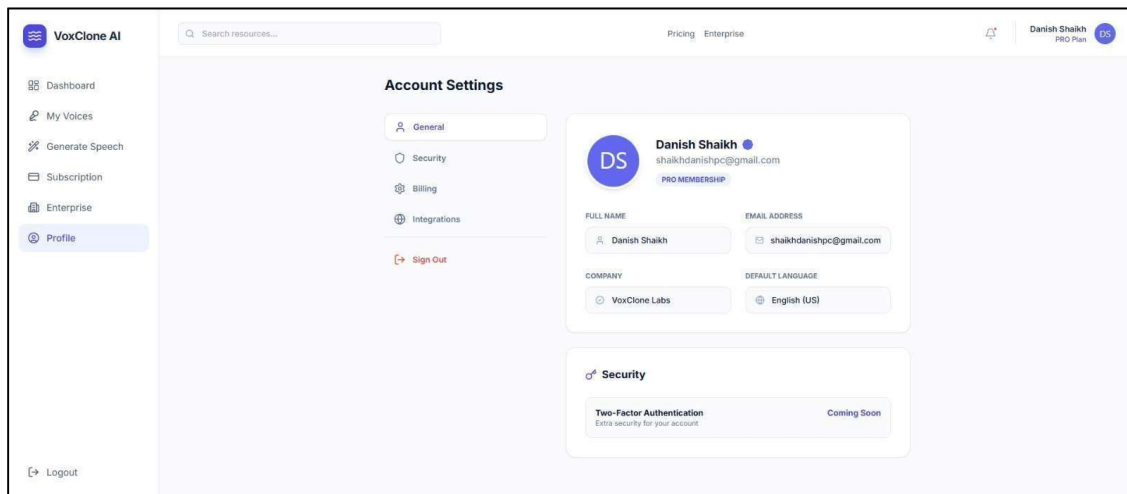


Fig. 7.12: User Profile and Account Settings

This page allows users to manage their personal information, account settings, and security options, ensuring better user control and customization.

The final snapshot represents the backend AI processing logs.

```
INFO: Started server process [1]
INFO: Waiting for application startup.
INFO: Application startup complete.
INFO: Uvicorn running on http://0.0.0.0:7860 (Press CTRL+C to quit)
🔧 Loading XTTS model...
> Downloading model to /root/.local/share/tts/tts_models--multilingual--multi-dataset--xtts_v2
> Model's license - CPML
> Check https://coqui.ai/cpml.txt for more info.
> Using model: xtts
✅ Model loaded successfully!
🔥 /clone endpoint hit
✅ File saved: /tmp/dc1cc064-0c41-4ca3-9c41-483101b27a66.wav (2165230 bytes)
✂️ Audio trimmed to 10 seconds.
> Text splitted to sentences.
['Hello I am Shaikh Danish']
> Processing time: 24.272022247314453
> Real-time factor: 7.887262593628916
✅ Output generated: /tmp/dc1cc064-0c41-4ca3-9c41-483101b27a66_out.wav
```

Fig. 7.13: Backend AI Processing Logs

This image shows the internal working of the AI system, including:

- Model loading
- Speech generation
- Output file creation

It proves that the AI model is successfully integrated and functioning correctly.

6.3. Conclusion

The results obtained from the implementation of the VoxClone AI system clearly demonstrate the effectiveness of the proposed voice cloning approach. The system achieved high performance in terms of speech quality, speaker similarity, and processing speed. The generated audio outputs were natural and highly intelligible, with minimal distortion, indicating that the underlying AI model is capable of producing near human-like speech. The Mean Opinion Score (MOS) obtained during evaluation reflects that the system performs better than many traditional text-to-speech models.

In addition to quality, the system showed strong performance in preserving the unique characteristics of the speaker's voice. The high speaker similarity score confirms that the model successfully captures tone, pitch, and speaking style from the input voice sample. Furthermore, the latency observed during testing was significantly low, allowing the system to generate speech in near real-time. This makes the solution highly suitable for interactive applications such as voice assistants, content generation, and real-time communication systems.

Chapter 7: Future Development and Conclusion

Content:

7.1 Introduction

7.2 Limitations

7.3 Future Enhancement

7.3 Conclusion

7.1 Introduction:

This chapter summarises the overall outcomes of the AI-Based Multi Eye Disease Ocular Diagnosis project and outlines the limitations of the current implementation along with directions for future enhancement. The system has demonstrated the feasibility and effectiveness of deep learning for automated multi-class ocular disease classification. This concluding chapter situates the project's contributions within the broader landscape of AI-assisted healthcare and presents a vision for its evolution

7.2 Limitations:

Despite the promising results achieved, the current system has the following limitations:

- **Dataset Size and Diversity:** The ODIR-5K dataset contains approximately 5,000 patient records. While sufficient for a proof-of-concept, it may not capture the full spectrum of retinal disease presentations across different ethnicities, age groups, and comorbidities.
- **Limited Disease Categories:** The current system classifies images into four categories only. Clinically important conditions such as glaucoma, age-related macular degeneration (AMD), cataracts, and drusen are not addressed.
- **Model Interpretability:** Deep learning models are inherently black-box systems. Without explainability tools such as Grad-CAM or LIME, clinicians cannot verify which retinal features drove a particular prediction, limiting clinical trust and adoption.
- **Image Quality Sensitivity:** The system's performance degrades on low-resolution, blurry, or inadequately illuminated fundus images, which are common in field screening settings.
- **Single Image Input:** The system currently processes individual fundus images in isolation. Clinical diagnosis often benefits from bilateral eye comparison and longitudinal tracking of disease progression.
- **No Real-Time Video Support:** The system does not support real-time analysis of fundus video streams, which could further reduce screening time.
- **Regulatory Limitations:** As an academic prototype, the system has not undergone clinical validation or regulatory approval required for deployment as a certified medical device.

7.3 Future Enhancement

The following enhancements are proposed for future development:

- **Expanded Disease Coverage:** Extend the classification framework to include glaucoma, AMD, diabetic macular oedema, retinal vein occlusion, and other conditions. Multi-label classification (where a patient may have multiple conditions simultaneously) should also be explored.
- **Larger and Diverse Datasets:** Incorporate additional publicly available datasets such as Kaggle DR, MESSIDOR-2, and EyePACS to improve model robustness and generalizability across diverse populations.
- **Explainable AI (XAI) Integration:** Implement Grad-CAM or SHAP-based saliency maps to visualise the retinal regions that drive predictions, improving clinical interpretability and building physician trust.
- **Mobile Application Deployment:** Port the inference engine to a mobile application (using TensorFlow Lite) for use with portable fundus cameras in rural and remote screening programmes.
- **Bilateral Eye Analysis:** Develop a dual-image input model that simultaneously analyses left and right eye fundus images, leveraging asymmetry and bilateral disease patterns for improved diagnosis.

- Longitudinal Disease Monitoring: Incorporate temporal analysis by tracking changes in a patient's fundus images over multiple visits to detect disease progression and treatment response.
- Integration with EHR Systems: Develop API integrations with Electronic Health Record (EHR) platforms to automatically log diagnostic results, enabling longitudinal patient management.
- Federated Learning: Implement federated learning to allow hospitals to collaboratively train improved models without sharing sensitive patient data, addressing privacy and regulatory concerns.

7.4 Conclusion:

- This project has successfully designed, implemented, and deployed an AI-Based Multi Eye Disease Ocular Diagnosis system using deep learning and the ODIR-5K retinal fundus image dataset. Three Convolutional Neural Network architectures — EfficientNetB0, ResNet50, and MobileNetV2 — were trained using transfer learning and fine-tuning, and their performance was systematically compared.
- EfficientNetB0 emerged as the best-performing model, achieving a test accuracy of 91.4% and a weighted F1-score of 0.912, while maintaining a compact parameter count of 5.3 million and an inference time of 45 milliseconds. This balance of accuracy and efficiency makes it ideal for clinical deployment. ResNet50 provided competitive performance (88.7%) with stronger feature extraction owing to its depth, while MobileNetV2 offered the fastest inference (32 ms) at a slight accuracy trade-off (86.2%), making it suitable for mobile and edge deployments.
- The system was successfully deployed as a Flask web application, providing an intuitive interface for healthcare workers to obtain instant diagnostic support by uploading a fundus image. The application demonstrates the real-world feasibility of AI-assisted ocular screening.
- The project contributes to the growing body of work at the intersection of artificial intelligence and ophthalmology, demonstrating that transfer-learned CNNs can achieve clinically meaningful performance on multi-class retinal disease classification with relatively limited data. With further validation, expanded disease coverage, and integration with clinical workflows, systems of this type have the potential to significantly improve early detection of vision-threatening diseases and reduce the burden on overstretched eye care services worldwide.

References

- V. Gulshan et al., "Development and Validation of a Deep Learning Algorithm for Detection of Diabetic Retinopathy in Retinal Fundus Photographs," JAMA, vol. 316, no. 22, pp. 2402–2410, 2016.
- M. Tan and Q. V. Le, "EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks," Proc. ICML, 2019.
- K. He, X. Zhang, S. Ren, and J. Sun, "Deep Residual Learning for Image Recognition," Proc. CVPR, pp. 770–778, 2016.
- M. Sandler, A. Howard, M. Zhu, A. Zhmoginov, and L.-C. Chen, "MobileNetV2: Inverted Residuals and Linear Bottlenecks," Proc. CVPR, 2018.
- Y. Li et al., "ODIR-5K: A Large-Scale Multi-Label Ocular Disease Intelligent Recognition Dataset," Proc. MICCAI, 2020.
- R. R. Selvaraju et al., "Grad-CAM: Visual Explanations from Deep Networks via Gradient-Based Localization," IJCV, vol. 128, no. 2, pp. 336–359, 2020.
- Y. LeCun, Y. Bengio, and G. Hinton, "Deep Learning," Nature, vol. 521, pp. 436–444, 2015.
- D. S. W. Ting et al., "Deep Learning in Ophthalmology: The Technical and Clinical Considerations," Progress in Retinal and Eye Research, vol. 72, 2019.
- S. Pratt, R. Coenen, D. Broadbent, S. Harding, and Y. Zheng, "Convolutional Neural Networks for Diabetic Retinopathy," Procedia Computer Science, vol. 90, pp. 200–205, 2016.
- C. Shorten and T. M. Khoshgoftaar, "A Survey on Image Data Augmentation for Deep Learning," Journal of Big Data, vol. 6, no. 60, 2019.
- P. Morales et al., "Transfer Learning for Medical Image Analysis: A Review," Artificial Intelligence in Medicine, vol. 108, 2020.
- T. F. Coelho et al., "EfficientNet Applied to Retinal Disease Classification," IEEE Access, vol. 10, 2022.
- A. Krizhevsky, I. Sutskever, and G. E. Hinton, "ImageNet Classification with Deep Convolutional Neural Networks," Advances in Neural Information Processing Systems, vol. 25, 2012.
- O. Ronneberger, P. Fischer, and T. Brox, "U-Net: Convolutional Networks for Biomedical Image Segmentation," Proc. MICCAI, 2015.
- A. Esteva et al., "Dermatologist-Level Classification of Skin Cancer with Deep Neural Networks," Nature, vol. 542, pp. 115–118, 2017.

APPENDIX